```
'default': {
'ENGINE': 'django.db.backends.postgresql',
'NAME': 'production_db',
'HOST': '127.0.0.1',
'PORT': '6432', # PgBouncer port
'CONN_MAX_AGE': 600,
}
}
```

Advanced query techniques for complex scenarios

Sometimes Django's ORM cannot express complex queries efficiently. Raw SQL provides full control:

```
from django.db import connection
def get_top_products(category_id, limit=10):
     with connection.cursor() as cursor:
       cursor.execute("""
           SELECT p.id, p.name, SUM(oi.quantity) as total_
sold
           FROM products_product p
           JOIN orders_orderitem oi ON p.id = oi.product_id
           WHERE p.category_id = %s
           GROUP BY p.id, p.name
           ORDER BY total_sold DESC
           LIMIT %s
           """, [category_id, limit])
     return cursor.fetchall()
```

Building a robust caching strategy

Caching stores frequently accessed data in fast storage—usually RAM—to avoid expensive recalculations or database queries. Effective caching can reduce database load by 70-90% in read-heavy applications.

- **Application-level caching** stores query results and API responses in memory using Redis or Memcached. This offers maximum control and microsecond access times.
- **HTTP caching** leverages browser and proxy caches using proper headers. Static content can be cached for hours or days.
- **Database query caching** stores SELECT query results. Application-level caching provides better control than database-internal caches.
- **Template fragment caching** caches rendered HTML portions. For expensive template tags, this dramatically improves response times.
- **Full-page caching** stores complete HTTP responses. For rarely changing pages, this offers maximum performance.

Implementing Redis for high-performance caching

Redis is an advanced in-memory data store that excels at caching. Unlike simple key-value stores, Redis supports complex data types like lists, sets, sorted sets, and hashes. Its sub-millisecond latency makes it ideal for high-traffic microservices.

Installing Redis on Ubuntu is straightforward using apt-get. The redis-server package includes everything needed. After installation, integrate Redis with Django using the django-redis package, which provides a Django cache backend.

```
#Install Redis
sudo apt-get install redis-server
sudo systemctl start redis-server

#Install django-redis
pip install django-redis

#Configure in settings.py
CACHES = {
     'default': {
       'BACKEND': 'django_redis.cache.RedisCache',
       'LOCATION': 'redis://127.0.0.1:6379/1',
       'OPTIONS': {
          'CLIENT_CLASS': 'django_redis.client.
DefaultClient',
          },
       'TIMEOUT': 300,
            }
       }
```

The basic caching pattern checks the cache first. If data exists (cache hit), return it immediately. If not (cache miss), fetch from the database, store in cache for future requests, and then return the data. This simple pattern applies to countless scenarios: user profiles, product listings, API responses, and calculated statistics.

Django provides multiple caching approaches. The cache_page decorator caches entire view responses. Template fragment caching caches portions of rendered templates. The low-level cache API provides maximum flexibility for custom caching logic.

```
from django.views.decorators.cache import cache_page
@cache_page(60 * 15) #Cache for 15 minutes
def product_list(request):
     products = Product.objects.all()
     return JsonResponse({'products': list(products.
values())})
```

Advanced Redis patterns for microservices

Beyond basic caching, Redis enables sophisticated patterns for microservices. Rate limiting tracks API request counts